V. AUTOMATISATION

A- MACRO-COMMANDES

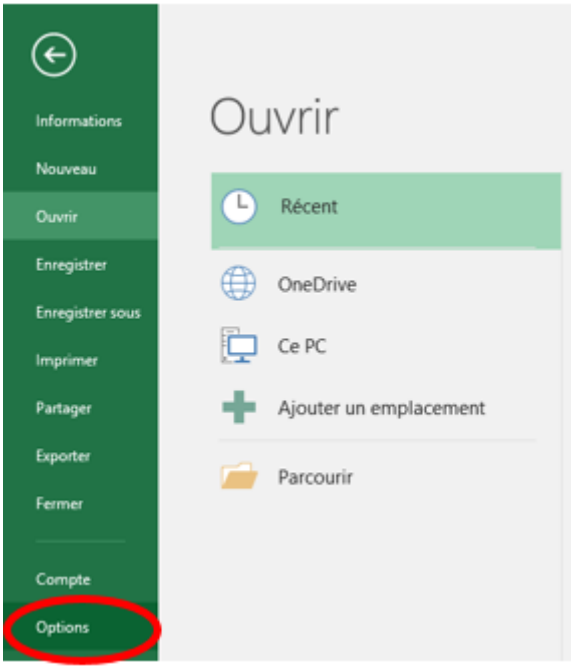
Lorsqu'on effectue fréquemment une tâche dans Microsoft Excel, on peut l'automatiser à l'aide d'une macro. Une macro est une série de commandes et de fonctions stockées dans un module Microsoft Visual Basic, qui sont exécutées l'une après l'autre dans un ordre défini.

Les macros vont donc permettre d'automatiser tout un tas de tâches comme par exemple:

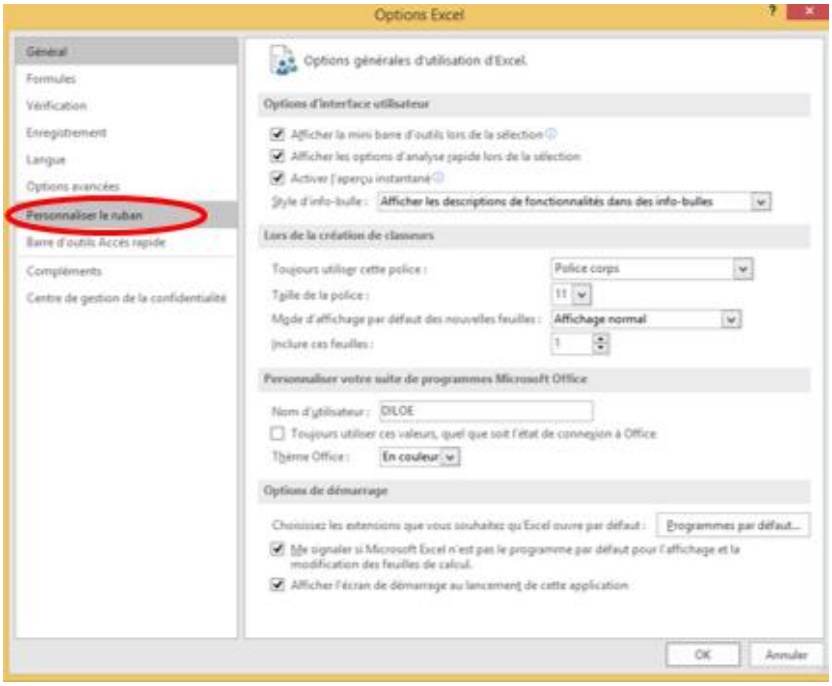
- Appliquer une police de texte : Mettre en gras, appliquer une taille de texte, mettre en italique, etc.
- Insérer une ligne automatiquement,
- Copier, coller des données automatiquement,
- Détecter des données, les additionner,
- Créer des feuilles, etc…

1- AFFICHAGE DE L'ONGLET DEVELOPPEUR

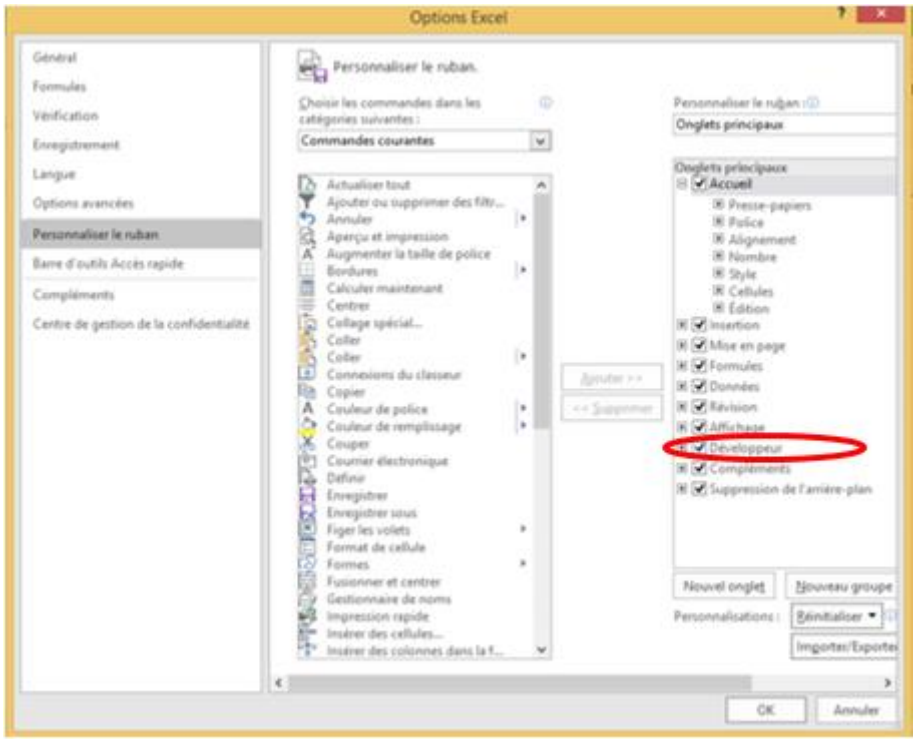
Avant toute chose, il est important d'afficher l'onglet « **Développeur** » dans le ruban pour accéder aux fonctions de programmation VBA. Pour y parvenir,



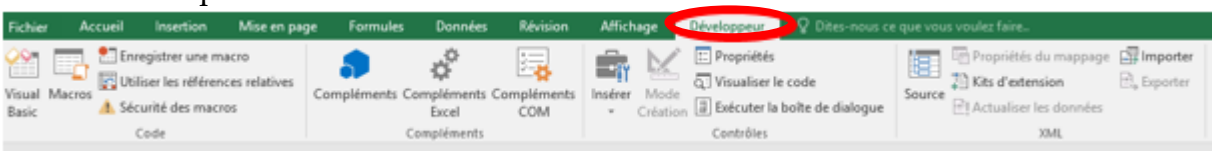
Cliquer sur l'onglet « **Fichier** » → cliquer sur la commande « **Option** ».



Sélectionner « **Personnaliser le ruban** » dans la fenêtre Option Excel.



Cocher la case « **Développeur** » de la liste « **Onglets principaux** ».
Valider en cliquant sur « **OK** ».

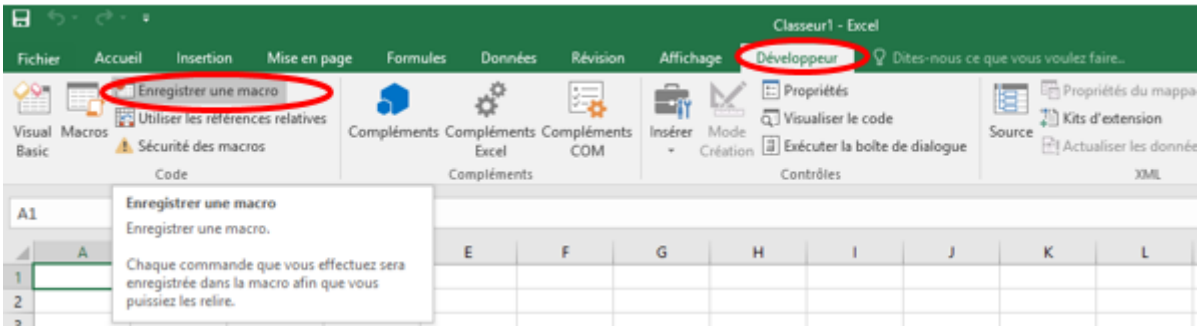


L'onglet « **Développeur** » apparait sur le ruban.

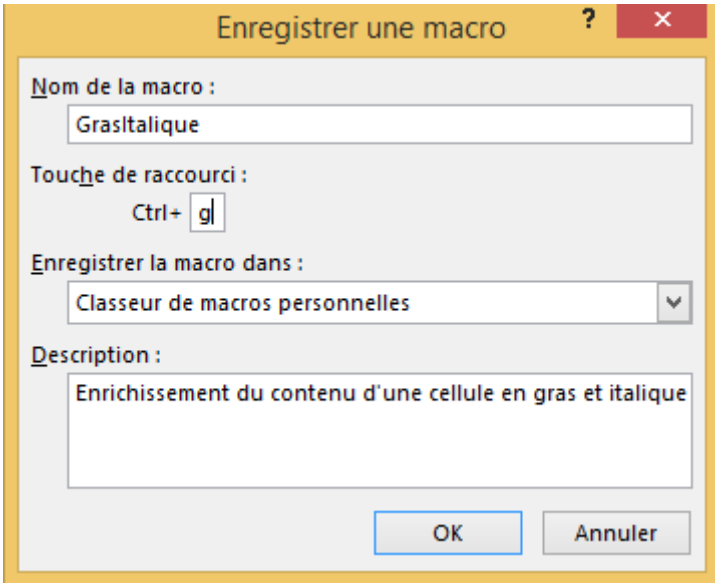
2- CREATION D'UNE MACRO

Nous allons créer une macro-commande qui permettra d'enrichir le contenu d'une cellule d'attributs de caractères (gras et italique). Pour y parvenir,

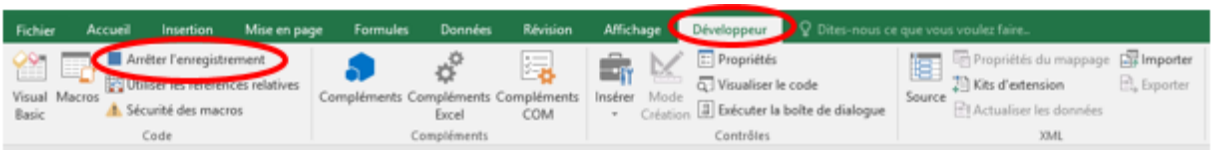
- Positionner le curseur sur une cellule à laquelle les formats de caractères voulus seront attribués. En exemple la cellule A1 ;



- Sélectionner la commande « **Développeur → Enregistrer une macro** » afin de lancer l'enregistrement de la macro ;



- Enfin, cliquer sur « **OK** ».



Le libellé du bouton « **Enregistrer une macro** » devient « **Arrêter l'enregistrement** », indiquant que la macro est en cours d'enregistrement.

- Sélectionner « **Gras (G)** » et « **Italique (I)** » du groupe « **police** » de l'onglet « **Accueil** ».
- Cliquer sur le bouton d'arrêt d'enregistrement de la macro.

Remarque : Le choix de « **Classeur macro personnelles** » permet à la macro d'être disponible à chaque utilisation d'Excel.

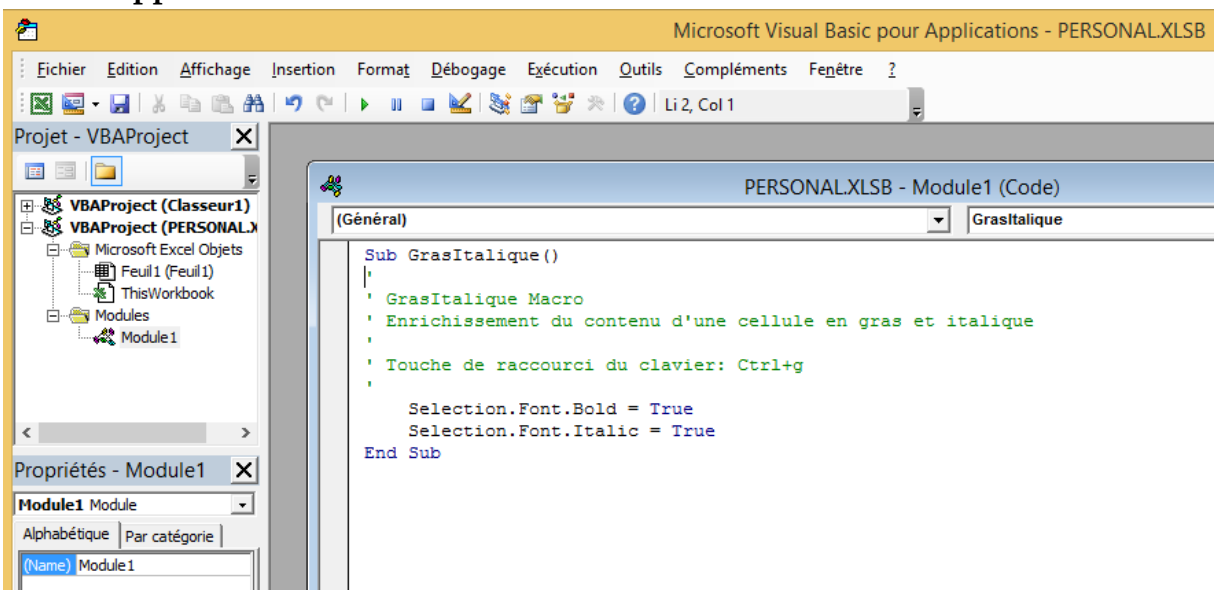
3- EXECUTION DE MACRO

La macro-commande s'exécute soit par :

- L'utilisation de raccourci clavier attribué lors de la création.
- La sélection du nom de la macro dans la boîte affichée par la commande « **Développeur → macros** » et le clique sur le bouton « **Exécuter** »
- L'utilisation du bouton affecté spécialement à la macro.

4- MODIFICATION D'UNE MACRO

Lors de la création d'une macro, une série de commandes et de fonctions sont stockées dans un module **Microsoft Visual Basic**. Elles sont exprimées dans un code qui peut être consulté, corrigé ou modifié dans l'éditeur « **Visual Basic Editor** » en cliquant sur « **Développeur → macros** ».

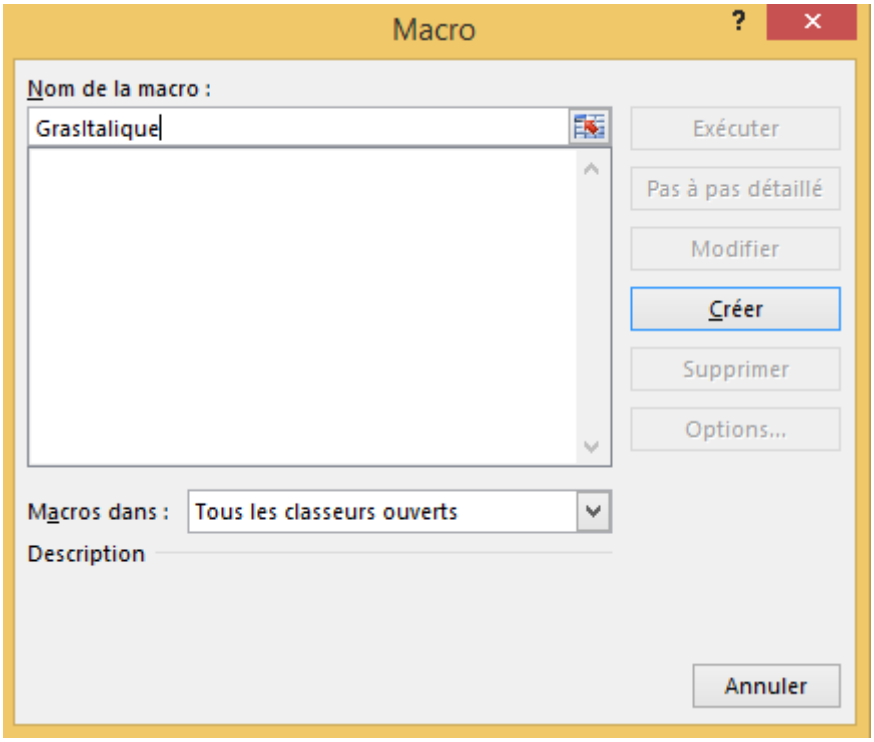


Remarque : L'affichage du classeur « **PERSONAL.XLSB** » par la commande « **Affichage → Afficher** » est obligatoire avant de procéder à la modification d'une macro.

5- ECRIRE UNE MACRO

Il est possible d'écrire directement une macro, sans l'aide de l'enregistreur de macro. Par exemple, pour écrire la macro « **GrasItalique** » :

- Cliquer sur le bouton « **Macros** » de l'onglet « **Développeur** ».
- Saisir le nom de la macro « **GrasItalique** » dans la zone « **nom de la macro** ».



- Cliquer sur le bouton « **Créer** ».
- Visual Basic Editor s'ouvre sur la fenêtre de code avec le texte ci-après :

```
Sub GrasItalique()

End Sub
```

- Saisir le texte de la macro entre « **Sub GrasItalique()** » et « **End Sub** ».
- Sélectionner la commande « **Fichier → Enregistrer sous** » pour enregistrer le classeur.
- Fermer et retourner dans Microsoft Excel.

Remarque : Contrairement à la méthode de l'enregistrement, la création d'une macro ne permet pas l'attribution d'un raccourci clavier. Pour affecter un raccourci clavier à la macro « **GrasItalique** », il faut procéder comme suit :

- Cliquer sur le bouton « **Macros** » de l'onglet « **Développeur** ».
- Sélectionner la macro « **GrasItalique** »
- Cliquer sur le bouton « **Options** ».
- Indiquer un raccourci clavier et saisir éventuellement une description pour la macro dans la boîte de dialogue « **Option de macro** ».
- Valider en cliquant sur « **OK** »

B- PROGRAMMATION EN VBA

Excel offre des possibilités très étendues. Pourtant, quelle que soit la puissance des fonctions d'Excel, elles ne peuvent répondre à toutes les situations. La programmation VBA 'Visual Basic pour Application) est la solution de personnalisation offerte par Excel, afin d'ajouter des caractéristiques, des fonctions et des commandes qui répondent précisément à vos besoins.

1- LES VARIABLES

La variable permet de stocker les informations de votre choix à tout moment de l'exécution d'un programme et de les réexploiter à n'importe quel moment.

Une variable peut être déclarée implicitement en lui affectant une valeur dans le programme. Mais il est souhaitable de déclarer une variable explicitement avant de l'utiliser.

- L'instruction « **OPTION EXPLICIT** », placée dans la section déclaration de la fenêtre code du module, exige à ce qu'une variable soit déclarée pour éviter les risques d'erreur liées à une faute de frappe dans le nom d'une variable.
- L'instruction Dim <nomvariable> As <type> permet de déclarer une variable.

Les différents types sont :

- **String** pour chaîne de caractère.
- **Integer** et **Long** pour les entiers courts et longs (i. e. de grande taille).
- **Single** et **Double** pour les réels.
- **Date** pour les dates.
- **Boolean** pour les valeurs logiques (booléennes).
- **Variant** pour le valeurs de tout type.

Les différents opérateurs sont :

- **&** permet la concaténation de deux chaines de caractères.
- / permet la division réelle.
- \ permet la division euclidienne, c'est-à-dire retourne la partie entière du résultat de la division.
  Exemple : 5\3 donne 1.

2- INSTRUCTION D'AFFICHAGE

MsgBox (<Expression>, [Boutons], [Titre de la boîte])

Expression, Boutons, Titre de la boîte sont des arguments nommés.

- **Expression** est un argument de type « **string** », correspondant au message affiché dans la boîte de dialogue.
- **Boutons** est un argument de type « **numérique** » facultatif. Il détermine les boutons affichés dans la boîte de dialogue, le symbole indiquant le type de message (information, question, etc.) et le bouton par défaut. Si cet argument est omis, un seul bouton « **OK** » s'affiche et aucune icone n'indique le type de message.
- **Titre de la boîte** est un argument de type « **string** » facultatif, correspondant au titre de la boîte de dialogue (affiché dans la barre de titre).
  Exemple : MsgBox ("Bonjour le monde ») affiche à l'écran Bonjour le monde.

3- INSTRUCTION D'ENTREE

<Identificateur variable> = InputBox (<Message>)

Cette instruction permet l'affichage d'une boîte de dialogue présentant une zone de texte dans laquelle l'utilisateur est invité à entrer des informations. L'information entrée est stockée dans l'identificateur variable.

Exemple : Age = InputBox("Donner l'âge de votre fils aîné :")

4- INSTRUCTION D'AFFECTATION

<Identificateur variable> = <Expression>

Cette instruction permet de stocker dans l'identificateur variable le résultat de l'évaluation de l'expression.

Exemple : Salmens = 1500 * 25

EXERCICE D'APPLICATION 1

Rédiger un programme VBA qui permet de calculer le montant net à payer par un client ayant acheté un seul produit en une quantité donnée. Le taux de la TVA en vigueur est 18%.

SOLUTION

```
Sub Facture()
   Dim Prix, Qte, MontBrut As Integer
   Dim montNet As Single
   Prix = InputBox("Prix unitaire de produit?")
   Qte = InputBox("Quantité achetée?")
   MontBrut = Prix * Qte
   montNet = MontBrut * 1.18
   MsgBox ("Montant à payer est " & montNet)
End Sub
```

5- INSTRUCTION CONDITIONNELLE (STRUCTURE ALTERNATIVE)

If <Condition> Then

<Action 1>

[Else

<Action 2>]

End If

<Action 1> ou <Action 2> est un ensemble d'instructions pouvant contenir également une autre instruction **If**

<Condition>, dans sa forme simple, se présente comme suit :

<Identificateur> <opérateur de comparaison> <Expression>.

Les opérateurs de comparaison sont :

Opérateur	Signification
=	Egal à
<	Inférieur à
>	Supérieur à
<>	Différent de
<=	Inférieur ou égal à
>=	Supérieur ou égal à

Lorsque la condition est vérifiée, l'action 1 est exécutée, puis dans un programme on passe à l'exécution de l'instruction qui vient après le mot End If. Dans le cas contraire l'action 2, si elle existe, est exécutée puis on passe à l'exécution de l'instruction qui vient après le mot End If.

EXERCICE D'APPLICATION 2

Composer un programme qui permet de calculer le net à payer par un client ayant acheté un produit en une quantité donnée. Il bénéficie d'une remise de 10% si le montant brut est supérieur à 100 000 F. Une taxe au taux égal à 18% est à payer.

SOLUTION

```
Option Explicit
Sub Netapayer()
   'Déclaration obligatoire des variables'
   Dim Prix, Qte As Single
   Dim MontBrut, MontNet As Double
   Prix = InputBox("Prix unitaire?")
   Qte = InputBox("Quantité achetée?")
   MontBrut = Prix * Qte
   If MontBrut > 100000 Then
      MontBrut = MontBrut * 0.9
   End If
   MontNet = MontBrut * 1.18
   MsgBox ("Le montant net à payer est " & MontNet)
End Sub
```

EXERCICE D'APPLICATION 3

La société « DECOR », spécialisée dans la vente des produits de décoration utilise Excel de Microsoft pour établir le relevé mensuel des salaires de ses agents. En plus de leur salaire, chaque salarié bénéficie d'une commission basée sur le chiffre d'affaire (CA) qu'il a réalisé dans le mois. Cette commission vaut :

- 1% du CA sur la tranche inférieure ou égale à 500 000 franc ;
- 2% du CA sur la tranche comprise entre 500 001 et 10 000 000 francs ;

4% du CA sur la tranche au-delà de 10 000 000 francs.

SOLUTION

```
Option Explicit
Sub Commission()
Dim CA, Commission As Single
CA = InputBox("Chiffre d'affaite?")
   If CA <= 500000 Then
      Commission = CA * 0.01
   Else
      If CA <= 10000000 Then
         Commission = 500000 * 0.01 + (CA - 500000) * 0.02
      Else
         Commission = 500000 * 0.01 + (10000000 - 500000) * 0.02 + (CA - 10000000) * 0.04
      End If
   End If
   MsgBox ("La commission est: " & Commission)
End Sub
```

6- INSTRUCTION D'ITERATION (LA BOUCLE TANT QUE)

While <Condition>
<Action>
Wend

Cette instruction permet de répéter l'action (**série d'instructions**) tant que la condition spécifiée est remplie. C'est l'une des structures les plus utilisées pour automatiser les tâches répétitives.

Lorsque la <**Condition**> spécifiée après « **While** » est réalisée, le programme exécute l'action, placée entre « **While** » et « **Wend** ». Lorsque l'instruction « **Wend** » est atteinte, le programme retourne à l'instruction « **While** » et interroge à nouveau la condition. Si elle est réalisée, l'action s'exécute à nouveau, etc. Dans le cas contraire, les instructions placées entre « **While** » et « **Wend** » sont ignorées, et l'exécution du programme se poursuit avec l'instruction située immédiatement après « **Wend** ».

EXERCICE D'APPLICATION 4

Composer un programme VBA qui permet de calculer le net à payer par un client ayant effectué un achat des produits. L'achat d'un produit est effectué en une quantité. Il bénéficie d'une remise de 10% si le total brut est supérieur à 100 000 frans. Une taxe aux taux égal à 18% est à payer. Le nom de chaque produit est saisi et le caractère $ comme nom du produit met fin à l'enregistrement des produits achetés.